CLOUD-06: GCP Integration

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 6 of 8 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

This notebook covers Dynatrace's integration with Google Cloud (GCP): the two connection models and where each is in its lifecycle, how each authenticates without long-lived keys, which services are covered, how to query GCP resources and Cloud Monitoring metrics with DQL, how to forward Cloud Logging into Grail, and how to monitor GKE and Cloud Run workloads.

Table of Contents

1. GCP Integration Architecture
2. Authentication
3. Supported GCP Services
4. Querying GCP Resources
5. GCP Metrics with DQL
6. Forwarding GCP Logs
7. GKE Monitoring
8. Cloud Run Analysis
9. Project, Folder and Label Governance
10. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail (the DQL cells do not run on Dynatrace Managed)
Permissions	`storage:metrics:read`, `storage:smartscape:read`, `storage:entities:read`, `storage:logs:read`, `storage:events:read`, `storage:spans:read`, `storage:buckets:read` to run the queries; `settings:objects:read` / `settings:objects:write` to manage connections
GCP	A project (or folder / organization) where you can create a service account and grant IAM roles
Connection	A GCP connection in the Clouds app (Preview) or the `dynatrace-gcp-monitor` deployment on GKE
Prior Knowledge	CLOUD-01 fundamentals; CLOUD-03 for the Kubernetes material in §7

1. GCP Integration Architecture

Dynatrace reads Google Cloud through two different integration models, and they are at different points in their lifecycle. Pick deliberately — they produce different resource models in Dynatrace (§4).

Connection Methods

Method	Lifecycle	Mechanism	Best For
GCP connection in the Clouds app	**Preview**	Dynatrace SaaS impersonates a service account in your project — no keys, nothing to deploy. Topology from Cloud Asset Inventory, plus metrics and logs	New GCP onboarding where Preview is acceptable

Method	Lifecycle	Mechanism	Best For
`dynatrace-gcp-monitor` on GKE	Maintenance mode	A container you run on GKE (Helm). It **polls** the Cloud Monitoring API for metrics and **pulls** logs from a Pub/Sub subscription	Production estates today; the working path until the Clouds-app connection reaches your tenant as a supported feature
OneAgent / OpenTelemetry	Current	OneAgent on GCE and GKE; OneAgent (Java, Node.js) or OTLP on Cloud Run	Code-level traces and process metrics — a different signal from the two rows above

> **Lifecycle, stated plainly.** The `dynatrace-gcp-monitor` README says: *"This integration is in maintenance mode. A new GCP poller is under active development."* The Clouds-app GCP connection is labelled Preview in Dynatrace's onboarding docs. Neither is a reason to wait: run the GKE monitor where you need a supported path now, and trial the Clouds-app connection on a non-production project. The older **Cloud Function** deployment of the monitor is gone — the README: *"It is now deprecated and has no support."* There is no ActiveGate-based GCP polling integration.

Data Flow

```
Clouds-app connection (Preview)
  Cloud Asset Inventory ──▶ Dynatrace SaaS   (topology: hourly scan + change
feed)
  Cloud Monitoring API  ──▶ Dynatrace SaaS   (metrics)

dynatrace-gcp-monitor on GKE
  Cloud Monitoring API ──(polled every 1–10 min, default 3)──▶ monitor ──▶
Dynatrace metrics API
  Cloud Logging ──▶ Log Router sink ──▶ Pub/Sub topic ──▶ subscription
──(pulled)──▶ monitor ──▶ Logs Ingest API
```

Pub/Sub carries **logs only**. Metrics are polled, never pushed through Pub/Sub — the monitor's metrics role grants `monitoring.timeSeries.list`, and its Helm values set the polling interval (*"Allowed values: 1 - 10"*, default `queryInterval: 3`).

GCP-Specific Considerations

- **Project-based organization** — projects, grouped into folders under an organization, are GCP's unit of ownership, billing and IAM
- **Labels, not tags** — GCP "labels" play the role of AWS/Azure tags
- **Cloud Monitoring API quotas** — the monitor's polling consumes the project's API quota; a shorter polling interval costs quota
- **Pub/Sub costs** — billed on bytes; filter at the Log Router sink (§6) before data ever reaches Pub/Sub

> **Sources:**
>
> - Google Cloud integration (DT docs)
> - GCP Cloud Platform Monitoring (DT docs) — page title *"GCP Cloud Platform Monitoring (Preview)"*
> - dynatrace-gcp-monitor README (Dynatrace GitHub) — *"This integration is in maintenance mode. A new GCP poller is under active development."*
> - dynatrace-gcp-monitor Helm values (Dynatrace GitHub) — *"Metrics polling interval in minutes. Allowed values: 1 - 10"*

2. Authentication

Both models avoid long-lived service account keys. Do not create or upload JSON keys for either.

Clouds-app Connection — Service Account Impersonation

1. **Create a service account** in the project that will host the connection.
2. **Grant it read roles** on everything it should see. The Terraform path in *Create your first GCP connection* grants `roles/viewer` on each listed project, `roles/resourcemanager.folderViewer` on each listed folder and `roles/resourcemanager.organizationViewer` on each listed organization. In manual mode you *"Grant permissions to the service account at project, folder, or organization level"*; the page's manual onboarding reference lists the roles.
3. **Allow Dynatrace to impersonate it.** Grant the Dynatrace principal for your environment (shown in the connection wizard)

`roles/iam.serviceAccountTokenCreator` **on that service account** — not on the project.

4. **Create the connection and its monitoring configuration** in the Clouds app, choosing locations and feature sets.

The same connection can be managed as code with the Terraform resources `dynatrace_gcp_connection` and `dynatrace_gcp_principal`. GCP IAM is eventually consistent, so a freshly granted role can take a minute or two to take effect — expect the first validation to retry.

`dynatrace-gcp-monitor` — Workload Identity

The monitor's deployment script creates **custom IAM roles** holding only what it needs — for metrics, among others `monitoring.timeSeries.list`, `compute.instances.list`, `cloudsql.instances.list`; for logs, `pubsub.subscriptions.consume` — and binds its Kubernetes service account to a GCP service account through **Workload Identity Federation for GKE** (`roles/iam.workloadIdentityUser`). The Dynatrace side uses an API token; the logs deployment needs `logs.ingest`.

Multi-Project Monitoring

Model	How it scales across projects
Clouds-app connection	Grant the impersonated service account viewer roles at folder or organization level; one connection covers everything beneath
`dynatrace-gcp-monitor`	Configure a Cloud Monitoring **metrics scope** that aggregates the projects, and set `scopingProjectSupportEnabled: true` so the monitor collects from all monitored projects

> **Best Practice:** Host the service account, Pub/Sub topics and the monitor in a **dedicated monitoring project**, so monitoring IAM and cost are separate from workload projects.
>
> **Sources:**
>
> - Create your first GCP connection (DT docs) — *"Grant `roles/viewer` on each listed project"*; *"Grant the Dynatrace principal `roles/iam.serviceAccountTokenCreator` on the new service account (impersonation trust)."*
> - dynatrace_gcp_connection (Dynatrace GitHub) — *"This resource uses service account*

> *impersonation, which requires the Dynatrace GCP Principal to be granted the* `roles/iam.serviceAccountTokenCreator` *role on the impersonated service account."*
>
> - dynatrace-gcp-monitor on GKE (DT docs)
> - dynatrace-gcp-monitor Helm values (Dynatrace GitHub)

3. Supported GCP Services

Category	Services
Compute	Compute Engine (GCE), Cloud Run functions (formerly Cloud Functions), App Engine
Containers	GKE (Google Kubernetes Engine), Cloud Run
Database	Cloud SQL, Cloud Spanner, Bigtable, Firestore
Storage	Cloud Storage, Filestore, Persistent Disk
Data	BigQuery, Pub/Sub, Dataflow, Dataproc
Networking	Cloud Load Balancing, Cloud CDN, Cloud DNS
AI/ML	Vertex AI

> **Where the authoritative list lives.** Coverage is defined per model and moves with each release: for `dynatrace-gcp-monitor`, the `gcpServicesYaml` block of its Helm values selects services, each backed by the Dynatrace **Google Cloud extension** (Extensions 2.0), which carries the metrics, topology rules and dashboards; for the Clouds-app connection, the feature sets offered in the monitoring configuration. Check the list in your own tenant rather than treating this table as exhaustive.

Metric latency is bounded by the polling interval (§1) plus Cloud Monitoring's own publication delay, which varies by metric — Google documents the sampling and visibility delay on each metric's reference entry.

> **Sources:** dynatrace-gcp-monitor Helm values (Dynatrace GitHub), Google Cloud integration (DT docs).

4. Querying GCP Resources

The two connection models create **different resource models**, so the query depends on which one feeds your tenant:

Model	Resource model	Query with
Clouds-app connection	Smartscape nodes named `GCP_<SERVICE_API>_<RESOURCE>` — for example `GCP_COMPUTE_GOOGLEAPIS_COM_INSTANCE`, `GCP_RUN_GOOGLEAPIS_COM_SERVICE`, `GCP_K8S_IO_POD`	`smartscapeNodes`
`dynatrace-gcp-monitor`	Classic generic entities from the Google Cloud extension — `dt.entity.cloud:gcp:gce_instance`, `…:cloudsql_database`, `…:cloud_run_revision` (see CLOUD-01 for the full list)	`fetch`

GCP Smartscape nodes carry `gcp.project.id`, `gcp.region`, `gcp.zone`, `gcp.organization.id`, `gcp.resource.name` and a `gcp.object` JSON blob with the full resource configuration.

What GCP Resources Does This Tenant Know About?

```
// Every GCP resource type discovered through the Clouds-app connection, with
counts
smartscapeNodes "GCP_*"
| summarize resource_count = count(), by:{type}
| sort resource_count desc
```

List Compute Engine Instances

```
// Compute Engine VMs (Clouds-app connection), with project and placement
smartscapeNodes "GCP_COMPUTE_GOOGLEAPIS_COM_INSTANCE"
| fields name, gcp.project.id, gcp.region, gcp.zone
| sort name asc
| limit 20
```

```
// Compute Engine VMs (dynatrace-gcp-monitor) — classic generic entity; the colon in the
// type name requires backticks. dt.entity.* is a look-back view, so it needs from:.
// The type exists only once the Google Cloud extension has created it. On a tenant without the
// monitor, the query carries a "The entity type ... wasn't found" warning
// (ENTITY_DATA_OBJECT_UNDEFINED) and returns zero rows — that means "not set up", not "no VMs".
fetch `dt.entity.cloud:gcp:gce_instance`, from:-7d
| fieldsKeep id, entity.name
| sort entity.name asc
| limit 20
```

Machine Type from the Resource Configuration

`gcp.object` holds the Cloud Asset Inventory record. Parse it for any configuration attribute the node does not expose as a field.

```
// Compute Engine instances by machine type (parsed from gcp.object)
smartscapeNodes "GCP_COMPUTE_GOOGLEAPIS_COM_INSTANCE"
| parse gcp.object, "JSON:gcpjson"
| fieldsAdd machineType = gcpjson[configuration][resource][machineType]
| summarize instance_count = count(), by:{machineType}
| sort instance_count desc
```

List Kubernetes Clusters (Including GKE)

```
// List all Kubernetes clusters (GKE clusters appear here)
fetch dt.entity.kubernetes_cluster, from:-7d
| fieldsKeep id, entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fieldsKeep id, name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer entities than
// the classic entity store; for a pre-migration inventory keep the classic query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

Kubernetes Workload Count

`dt.entity.cloud_application` is the classic Kubernetes **workload** entity — it counts GKE workloads alongside workloads on any other monitored cluster. Cloud Run is **not** in it: Cloud Run instances monitored by OneAgent appear as hosts (§8).

```
// Kubernetes workloads (all monitored clusters, GKE included)
fetch dt.entity.cloud_application, from:-7d
| summarize workload_count = count()
```

5. GCP Metrics with DQL

Cloud Monitoring metrics land in Grail under `cloud.gcp.*`. For `dynatrace-gcp-monitor`, the key is built from the Google metric type: `compute.googleapis.com/instance/cpu/utilization` becomes `cloud.gcp.compute_googleapis_com.instance.cpu.utilization` (dots in the service domain become underscores; `/` becomes `.`). Discover what your tenant actually holds before building on a key — a `timeseries` against a key that does not exist returns an empty result, not an error.

> **Sources:** dynatrace-gcp-monitor HACKING.md (Dynatrace GitHub) — *"timeseries avg(cloud.gcp.compute_googleapis_com.instance.cpu.utilization)"*; Compute Engine monitoring (DT docs) lists the source metric `compute.googleapis.com/instance/cpu/utilization`.
>
> **Derived:** the naming rule follows from the two forms side by side.

```
// Discover the GCP metric keys present in this tenant (note: no comma before
from:)
metrics from:now()-2h
| filter startsWith(metric.key, "cloud.gcp.")
| summarize n = count(), by:{metric.key}
| sort metric.key asc
```

```
// Compute Engine CPU utilization (0–1 ratio) over the last 6 hours
timeseries cpu =
avg(cloud.gcp.compute_googleapis_com.instance.cpu.utilization), from:-6h
| fieldsAdd avgCpuPct = arrayAvg(cpu) * 100
```

6. Forwarding GCP Logs

Cloud Logging does not push to Dynatrace directly. Every GCP log path starts at the **Log Router**: a **sink** selects log entries with an inclusion filter (minus exclusion filters) and routes them to a Pub/Sub topic, from which the integration reads.

The `dynatrace-gcp-monitor` Log Path

1. **Create a Pub/Sub topic and subscription** in the monitoring project. The monitor's deployment script creates the subscription with a 120-second acknowledgement deadline and one day of message retention, and its Helm deployment rejects any other ack deadline.
2. **Create a Log Router sink** with the topic as destination and an inclusion filter for exactly what you want.
3. **Grant the sink's writer identity `roles/pubsub.publisher` on the topic.** Until you do, nothing arrives: *"Until you grant this identity write-access to the destination, log entry exports from this sink will fail."*
4. **Deploy the monitor in logs mode** with a Dynatrace token holding `logs.ingest`. It pulls the subscription, batches, and sends to the Logs Ingest API.

For the Clouds-app connection (Preview), follow the log setup in the connection's onboarding flow.

Sink Design Rules

Rule	Consequence
Organization and folder sinks export child projects only when `includeChildren` is set: *"If the field is true, then log entries from all the projects, folders, and billing accounts contained in the sink's parent resource are also available for export."*	One **aggregated sink** at the organization or folder replaces a sink per project — and one change point for filters
"If a log entry is matched by both `filter` and one of `exclusion_filters` it will not be exported."	Exclusions are the cheapest volume control: they act before Pub/Sub bills a byte

Rule	Consequence
"You cannot modify the _Required sink or exclude logs from it."	Admin Activity audit logs always stay in Cloud Logging; your Dynatrace sink is an additional copy
"Except for BigQuery, Data Access audit logs are disabled by default because they can generate large volumes of data."	Enable them deliberately per service — they are high volume, and a sink cannot export what is never written

> **Pub/Sub cost.** *"Pub/Sub service charges are based on usage (the number of published, delivered, or stored bytes)."* Every byte your sink admits is metered by Pub/Sub on the way through, before Dynatrace ingest is counted.

Sizing the Log Forwarder

The monitor's default logs container (1.25 vCPU, 1 GiB) handles about **8 GB of logs per hour**. Above that, messages accumulate in the Pub/Sub subscription rather than failing — back-pressure is silent until you watch the subscription backlog.

- **Scale horizontally** (more replicas). Dynatrace advises against vertical scaling, because each larger machine needs its process-count setting re-tuned to its cores — but not from the base 1.25 vCPU / 1 GiB size: move the container to 4 vCPU / 4 GiB first, then add replicas.
- **Autoscaling** is recommended only from about **450 MB/min** of throughput, on 4 vCPU / 4 GiB nodes.
- Size from the tested configurations on the logs-only setup page, using your **post-filter** volume. Each row is a 4 vCPU / 4 GiB machine with `PARALLEL_PROCESSES=4`, `NUMBER_OF_CONCURRENT_MESSAGE_PULL_COROUTINES=30` and `NUMBER_OF_CONCURRENT_PUSH_COROUTINES=20`:

Achieved throughput	Replicas
~8 MB/s (~480 MB/min)	1
~25 MB/s (~1.5 GB/min, ~2 TB/day)	4
~46 MB/s (~2.7 GB/min, ~4 TB/day)	6

Find GCP Logs in Grail

The monitor stamps each record with `cloud.provider` (`gcp`), `gcp.project.id`, `gcp.region`, `gcp.resource.type` (the Cloud Logging monitored-resource type) and `log.source` (the Cloud Logging `logName`). Filter on `gcp.resource.type` rather than on `cloud.provider` alone: host and container logs collected by OneAgent can carry the cloud provider too (this was observed on Azure hosts in this series' validation), so a provider-only filter risks mixing them into the result.

```
// GCP platform logs forwarded through the Log Router, by project and resource
type
fetch logs, from:-1h
| filter isNotNull(gcp.resource.type)
| summarize log_count = count(), by:{gcp.project.id, gcp.resource.type}
| sort log_count desc
| limit 20
```

Cloud Audit Logs get three more attributes — `audit.identity` (the calling principal), `audit.action` (the API method) and `audit.result` — which make "who changed what" a one-line query:

```
// Who is making administrative changes? (Cloud Audit Logs forwarded by the
monitor)
fetch logs, from:-24h
| filter contains(log.source, "cloudaudit.googleapis.com")
| summarize changes = count(), by:{audit.identity, audit.action}
| sort changes desc
| limit 20
```

If these return nothing, check in order: the sink's writer identity has `roles/pubsub.publisher`; the subscription has a backlog (logs are arriving but not being pulled); the monitor's token has `logs.ingest`; the query window. The field names above are those the monitor writes — the Clouds-app connection's log path may differ, so inspect one record before building on them.

> **Sources:**
>
> - dynatrace-gcp-monitor architecture (Dynatrace GitHub) — *"Pub/Sub messages with log entries are polled by containerized `dynatrace-gcp-monitor`, processed, batched and sent to Dynatrace Log Ingest API."*
> - Set up GCP log integration — logs only (DT docs) — *"Autoscaling is recommended only when you*

> *have a minimum of 450 MB/min throughput and can provide a 4vCPU 4Gi RAM machine."* The sizing table above is reproduced from this page.
>
> - Cloud Audit Logs overview (Google Cloud) — *"Except for BigQuery, Data Access audit logs are disabled by default because they can generate large volumes of data."*
> - logging_config.proto (Google GitHub) — *"Until you grant this identity write-access to the destination, log entry exports from this sink will fail."*
> - Pub/Sub pricing (Google Cloud) — *"Pub/Sub service charges are based on usage (the number of published, delivered, or stored bytes)."*
> - **Derived:** the log-field list and the audit attributes come from the monitor's log-processing configuration (`src/config_logs`) in the repository above.

7. GKE Monitoring

GKE runs the same Dynatrace Operator and DynaKube as EKS (CLOUD-03) and AKS (CLOUD-05). What is GKE-specific is Autopilot's admission model and identity.

GKE-Specific Considerations

Feature	Monitoring Impact
GKE Standard	No GKE-specific DynaKube configuration is needed for the Operator's deployment modes
GKE Autopilot	Autopilot admits privileged partner workloads only through a GKE **allowlist**. The Operator's Helm chart creates the `AllowlistSynchronizer` (`auto.gke.io/v1`) automatically when the cluster offers it; the allowlisted Dynatrace workloads are the **CSI driver**, **log monitoring** and the **CSI job**. On Autopilot the supported mode is App Observability: *"For GKE Autopilot, you can install Dynatrace for App Observability."*
Helm `platform: gke-autopilot`	Marked **deprecated** in the Operator chart — the platform is detected automatically; remove the override from existing values files

Feature	Monitoring Impact
Workload Identity Federation for GKE	Google's recommended way for pods to reach Google Cloud APIs — use it for anything in-cluster that calls GCP (including `dynatrace-gcp-monitor`) instead of mounting keys
Control plane	GKE's API server, scheduler and controller-manager metrics are sent to **Cloud Monitoring** when you enable them on the cluster (which must also have system metrics enabled); GKE audit logs are in Cloud Logging and reach Dynatrace through a sink (§6)

GKE Container Metrics

```
// Top 10 container names by CPU, averaged across every replica that shares
the name
timeseries containerCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.container.name}
| fieldsAdd avgCpu = arrayAvg(containerCpu)
| sort avgCpu desc
| limit 10
```

GKE Node Utilization

```
// Node CPU utilization across GKE nodes over the last hour, derived at container grain.
//
// `dt.kubernetes.node.cpu_usage` and `dt.kubernetes.node.memory_working_set` DO NOT EXIST
// (verified 08/12/2026 against a full 836-key catalog enumeration). The `dt.kubernetes.node.*`
// namespace is real, but it publishes CAPACITY and CONDITION metrics — `cpu_allocatable`,
// `memory_allocatable`, `pods_allocatable`, `conditions` — not utilization. Node-level
// utilization is derived by summing the container metrics per node, which is what this cell does.
// A timeseries against a missing key returns an EMPTY result rather than an error, so the old
// cell simply drew nothing. Enumerate the real catalog with:
//   metrics | filter startsWith(metric.key, "dt.kubernetes") | summarize n = count(), by:{metric.key} | sort metric.key asc
// (`metrics` takes `from:` with NO leading comma; `summarize` requires an aggregation, and a
//  bare `metrics | fields metric.key` is capped and will silently under-report the catalog.)
timeseries used = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-1h, by:{dt.entity.kubernetes_node}
| fieldsAdd avgCpu = arrayAvg(used)
| sort avgCpu desc
| limit 10
```

GKE Pod Events

```
// Kubernetes events by reason over the last 6 hours.
//
// Corrected 08/11/2026 — this cell previously carried three separate filters
that each
// matched nothing, so it returned zero rows against an estate emitting
thousands:
//   1. `event.kind == "K8S_EVENT"` — event.kind never takes that value.
Verified over 24h
//      the only values are DAVIS_EVENT, SYNTHETIC_EVENT, FLEET_EVENT and
DAVIS_PROBLEM.
//      Kubernetes events arrive as DAVIS_EVENT; the discriminator is
event.provider.
//   2. `event.type == "Warning"` — every KUBERNETES_EVENT record carries
CUSTOM_INFO
//      (3,571 of 3,571 checked). The Warning/Normal split is in status
("WARN" / "INFO");
//      dt.kubernetes.event.important was "true" on every event over 30 days
(10/02/2026).
//   3. `event.reason` — null on every record. The real field is
dt.kubernetes.event.reason.
// Each filter was valid syntax that executed cleanly, which is why this
survived review.
fetch events, from:-6h
| filter event.provider == "KUBERNETES_EVENT"
| filter status == "WARN"
| summarize event_count = count(), by:{dt.kubernetes.event.reason}
| sort event_count desc
| limit 10
```

Sources:

- Kubernetes supported technologies (DT docs) — *"For GKE Autopilot, you can install Dynatrace for App Observability."*
- Control plane metrics (Google Cloud) — *"The cluster must have system metrics enabled."*
- AllowlistSynchronizer template (Dynatrace GitHub)

8. Cloud Run Analysis

Cloud Run is GCP's serverless container platform. It shares some monitoring patterns with Lambda but runs containers instead of functions.

Cloud Run Monitoring Points

Metric	Description	Alert Threshold
Request count	Total HTTP requests	Baseline deviation
Request latency	Response time (p50, p95, p99)	SLO-dependent
Instance count	Active container instances	Max instance limit
CPU utilization	Per-instance CPU usage	> 80% sustained
Memory utilization	Per-instance memory	> 80% (risk of OOM)
Startup latency	Cold start time for new instances	Application-dependent

Cloud Run vs Lambda

Aspect	Cloud Run	AWS Lambda
Unit	Container	Function
Runtime	Any Docker image	Managed runtimes + custom
Max duration	Services: request timeout up to 60 minutes. Jobs: task timeout up to 7 days (1 hour for tasks using GPUs)	15 minutes
Concurrency	Multiple requests per instance (configurable, up to 1,000)	One per execution (default)
Monitoring	OneAgent (Java/Node.js only) or OTLP — an OpenTelemetry Collector can run as a sidecar container	Lambda Layer
Cold start	Container pull + startup	Runtime initialization

Sources:

- Request timeout (Google Cloud) — *"The timeout is set by default to 5 minutes (300 seconds) and can be extended up to 60 minutes (3600 seconds)."*
- Task timeout (Google Cloud) — *"For tasks using GPUs, the maximum available timeout is 1 hour."*
- About concurrency (Google Cloud) — *"You can increase this to a maximum of 1,000."*

Cloud Run Monitoring: OneAgent Coverage and Caveats

OneAgent monitoring of Cloud Run managed is **limited to Java and Node.js**. Cloud Run itself runs any container image, but OneAgent auto-injection does not cover every runtime — other runtimes (Go, Python, .NET) send telemetry via **OTLP** instead.

Aspect	Detail
Supported runtimes (OneAgent)	Java and Node.js only — other runtimes use OTLP-direct
Credentials	Baked in as **build-time image arguments** (`DT_API_URL` , `DT_API_TOKEN`) — repointing to a new tenant requires an image rebuild and a new revision
Execution environments	Gen1 vs Gen2 — **Gen1 does not expose CPU/memory metrics** (intentional security limits); Gen2 has no such restriction
Where instances appear	On the **Hosts** page (with GCP properties and each instance's memory limit), not the Container groups page
Cold-start overhead	Agent injection adds startup overhead; because each revision autoscales from zero, cold starts appear more often than on always-on hosts

Migration note: because Cloud Run credentials are build-time image args, redirecting a Cloud Run workload to a new Dynatrace tenant means rebuilding the image and deploying a new revision — see **M2S-05: Step 5 — Execute** for the full serverless/container redirect matrix.

Source: Monitor Google Cloud Run managed (DT docs)

```
// Cloud Run service spans in the last hour
//
// Corrected 08/12/2026: `cloud.platform` is DEPRECATED in the semantic
dictionary and carried no
// value on any span in the validation tenant, so `cloud.platform ==
"gcp_cloud_run"` could only
// ever return nothing. The dictionary marks `cloud.platform` "Deprecated, no
replacement available";
// `cloud.provider` (stable) is the nearest field, and narrows to provider
only (aws / azure / gcp / ...).
// Note this narrows to provider, not service — add a dt.service.name or
faas.name filter to isolate
// Cloud Run specifically. On a tenant with no GCP workloads this correctly
returns no rows.
fetch spans, from:-1h
| filter span.kind == "server" and cloud.provider == "gcp"
| summarize {avg_duration_ms = avg(duration) / 1ms, request_count = count()},
by:{dt.service.name}
| sort request_count desc
| limit 10
```

Tracing Across Pub/Sub

GCP shops frequently place Pub/Sub between services. **Pub/Sub is absent from OpenTelemetry's auto-instrumentation coverage** where Kafka and RabbitMQ are first-class (the OpenTelemetry Java supported-libraries list carries Kafka `0.11+` and RabbitMQ `2.7+`, but no Pub/Sub entry) — so an auto-instrumented application does not propagate trace context across Pub/Sub for you, and a distributed trace breaks silently at every hop. Coverage varies by language and client version; verify for your own runtime.

Two ways to close the gap:

- **Use the client's own tracing (Java).** Google's Pub/Sub Java client added OpenTelemetry tracing to the Publisher and Subscriber in version 1.133.0 — enable it with `setEnableOpenTelemetryTracing(true)` and pass your `OpenTelemetry` instance with `setOpenTelemetry(...)` on the builder. Check your language's client for the equivalent before hand-rolling propagation.
- **Propagate it yourself** where the client cannot. Carry the W3C **`traceparent`** through the message `attributes` map: the **publisher** injects the active trace context into `message.attributes` before publishing; the **subscriber** reads `traceparent` and uses it as the parent of the consumer span.

Without either, producer and consumer spans appear as disconnected traces. See the **SPANS** series for span and trace-context fundamentals.

> **Sources:** Span and trace context propagation (DT docs); OpenTelemetry Java supported libraries (OpenTelemetry GitHub); java-pubsub CHANGELOG (Google GitHub) — *"Add OpenTelemetry tracing to the Publisher and Subscriber"*; OpenTelemetry Collector on Cloud Run (Google Cloud GitHub)

9. Project, Folder and Label Governance

GCP's hierarchy — organization → folders → projects — arrives in Dynatrace as **fields on the data**, not as tags you have to propagate by hand.

GCP Construct	Where it appears in Dynatrace
Organization	`gcp.organization.id` on Smartscape nodes
Folder	Not a field — express folder ownership through labels, or through the projects it contains
Project	`gcp.project.id` on Smartscape nodes, forwarded logs and monitor metrics — the primary scoping unit
Region / Zone	`gcp.region`, `gcp.zone`
Labels	`` `tags:gcp_labels` `` on Smartscape nodes (backticks required)

Best Practices

Practice	Description
Scope by `gcp.project.id`	Build segments, dashboards and cost views on the project field rather than on free-text tags
Enforce label conventions at the source	Standard labels (`env`, `team`, `service`, `cost-center`) enforced by organization policy, so every resource and log carries them

Practice	Description
Segments for views, IAM for access	A segment per project is a good default view, but segments filter; they do not restrict access — *"Regardless of configured visibility, any segment can be accessed with storage:filter-segments:read permission."* Restrict access with IAM policies or bucket permissions
Route problems on ownership labels	Problem-triggered workflows keyed on `team` / `owner`, not on project names alone

> **Sources:** Visibility of segments (DT docs) — *"Regardless of configured visibility, any segment can be accessed with storage:filter-segments:read permission."*

10. Summary and Next Steps

Key Takeaways

- **Two models, two lifecycles:** the Clouds-app GCP connection (Preview; keyless service account impersonation, Cloud Asset Inventory topology) and `dynatrace-gcp-monitor` on GKE (maintenance mode; the working production path today). The Cloud Function deployment is unsupported
- **No keys in either model** — impersonation via `roles/iam.serviceAccountTokenCreator`, or Workload Identity Federation for GKE
- **Pub/Sub carries logs only**; metrics are polled from Cloud Monitoring and land under `cloud.gcp.*`
- **Query by model:** `smartscapeNodes "GCP_*"` for the Clouds-app connection, `fetch` `` `dt.entity.cloud:gcp:*` `` for the monitor
- **Logs start at a Log Router sink** — aggregate at folder/organization with `includeChildren`, cut volume with exclusions before Pub/Sub bills it, and grant the writer identity `roles/pubsub.publisher`
- **GKE Autopilot** admits the Operator's CSI driver through an allowlist the Helm chart creates for you; **Cloud Run** is OneAgent for Java/Node.js, OTLP (optionally via a Collector sidecar) for everything else
- **Govern on `gcp.project.id` and labels**; IAM for access, segments for views

Next Steps

- **CLOUD-07: CloudWatch Log Ingestion** — the AWS counterpart to §6
- **CLOUD-08: Multi-Cloud Patterns** — unified monitoring across GCP and other providers
- **K8S series** — DynaKube configuration in depth

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.